Live interactive demo · No login required to read this page

Union Records Management System

A guided tour of the build so far — Phases 1 through 5 complete (24 of 37 chunks). End-user scenarios, click-through links to real screens, and "what to look for" callouts so you know exactly what each feature demonstrates.

How to use this demo

You're on the public tour page. To interact with the live screens:

1. Pick a role below (Registrar / Inspector / Officer) and copy its credentials.
2. Click "Open login screen" → sign in with the copied email + password.
3. You'll land on the Dashboard. Use the sidebar to explore.
4. Each scenario card below has an "Open this screen" button — click after signing in.

Tip: If you reached this page via the chat UI's "Live Preview" or "Open in New Tab" button, you're already on the public URL — just sign in to start clicking through.

Phases 1-7 complete | ~20 min walkthrough | 614+ automated assertions | 6 demo unions · 39 documents

A. Foundation | B. Records | C. Compliance | D. Oversight | G. Productivity (Phase 6) | H. Public Portal (Phase 7) | E. Administration | F. Officer's scoped view

Start here: pick a role

Three demo accounts are pre-seeded. Click an email to copy it, then click "Open login" to sign in.

Registrar

Full system access. All screens. Use this for the admin scenarios.

```
registrar@example.com
```

Test!2345

Inspector

Oversight role. Sees Unions, Documents, Compliance, Compare, Audit, Retention (reject only). No admin screens.

inspector@example.com

Test!2345

Officer

Union-scoped. Sees only "My Union" (Papua Maritime) + own compliance. No global oversight.

officer@example.com

Test!2345

Open login screen | Go straight to dashboard (will prompt login)

A. Foundation

Authentication, 2FA, and session management — the bedrock every later phase builds on.

Sign in as a Registrar

2 min

You are the Registrar — the system administrator. From sign-in you land on the Dashboard, which is your daily command center. The login flow uses an HMAC-signed session cookie with a JTI (JWT ID) for revocation; wrong passwords and wrong 2FA codes share a single lockout counter, so 3 failures of any kind locks the account for 15 minutes.

TRY IT

1. Click "Open login screen" below.
2. Sign in with registrar@example.com / Test!2345.
3. You should land on /dashboard automatically.
4. Open the user menu (top-right) → "Active Sessions" to see this session listed with its JTI.

WHAT TO LOOK FOR

→ No flash of unauthenticated content — the App Shell waits for /api/auth/me before rendering the sidebar.

→ The "Active Sessions" dialog shows your current session with a "This device" badge — that badge is computed from the JTI match, not a magic flag.

→ Try signing in as inspector@example.com in another browser — note the nav sidebar shows different items (no Users, no Security, no API Keys).

Open login screen →

Enable 2FA on your account

3 min

Two-factor authentication uses TOTP (RFC 6238) — the same standard as Google Authenticator, Authy, 1Password. Setup happens in two phases: first you scan the QR code and verify a real code (only then is the secret persisted), then every future login requires both password and a fresh 6-digit code. Disabling 2FA requires your password; an admin can reset it (which also clears the secret).

TRY IT

1. Sign in as registrar@example.com.
2. Open the user menu (top-right) → "Account Security".
3. Click "Enable 2FA" → scan the QR code with an authenticator app.
4. Enter the 6-digit code from your app to confirm.
5. Sign out, then sign back in — you'll be prompted for a 2FA code after your password.

WHAT TO LOOK FOR

→ The QR code is generated server-side from a per-user secret stored in the DB; the secret is never sent back to the client in plaintext after setup.

→ After enabling, the login flow has TWO steps: password → 2FA challenge → authenticated session. The 2FA challenge is a short-lived opaque token (30s), not the session itself.

→ A wrong 2FA code increments the SAME lockout counter as a wrong password — 3 wrong attempts of any kind locks the account.

Open account security →

B. Records

Unions and documents — the core registry. Phase 2's 5 chunks built this end-to-end.

Browse the Union Registry

2 min

The Union Registry is the master list of registered trade unions — 6 demo unions are pre-loaded covering all 3 statuses (active, suspended, deregistered). Default sort is by document count descending. Click any union to see its full detail page.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Unions" in the sidebar.
3. Note the 6 demo unions (Papua Maritime, Highlands Coffee, Lae Stevedores, Southern Teachers, New Islands Fishers, Western Miners).
4. Click "Papua Maritime Workers Union" to open Union Detail.

WHAT TO LOOK FOR

→ The status badge on each row (Active = green, Suspended = yellow, Deregistered = gray).

→ Union Detail shows 13 document-type tiles, each with a real compliance badge (Compliant / Expiring / Overdue / Missing / N/A) — this is the loop-close from Phase 2 → Phase 3.

→ The Officer silent-pin rule: try editing a union as an Officer — address fields save, but unionName/registrationNo/status are silently ignored server-side.

Open union registry →

Upload a document with version history

4 min

Documents are filed against a union, typed as one of 13 canonical types (constitution, bylaws, annual return, etc.), and stored as files on disk under /storage/unions/{unionId}/{documentId}.bin. SHA-256 checksums detect duplicates. Uploading a new version of an existing document preserves the old version in a version chain — restoring an old version creates a NEW version (v4 with v1's content), never mutates history.

TRY IT

1. Sign in as registrar@example.com.
2. Open any union (e.g. Papua Maritime).
3. Scroll to a document-type tile → click "Upload".
4. Pick a PDF file, give it a title, submit.
5. After upload, click the document title → "Version history" to see v1.
6. Upload another file to the same type — note the version number increments.

WHAT TO LOOK FOR

→ The upload is atomic: file written to .tmp first, then renamed — a crash mid-write never produces a partial file.

→ The checksum is shown in the document detail panel — uploading the same file again to the same union + type triggers a duplicate-detection warning.

→ Version history shows every version with its uploader, timestamp, and file size. Restoring v1 of a v3 doc creates v4 (with v1's content) — the original v1 row is never mutated.

Open first union →

C. Compliance

The highest-leverage principle of Phase 3: compute live, never store. Every status is recomputed on read.

Review compliance across all unions

3 min

The Compliance screen shows every union × every document type in a single matrix. Statuses are computed live via pure functions (no DB caching) — so editing a rule instantly changes every union's score. The 5 statuses: Compliant (green), Expiring (yellow, ≤90 days to renewal), Overdue (red), Missing (gray), Not Applicable (excluded from scoring).

TRY IT

1. Sign in as registrar@example.com.
2. Click "Compliance" in the sidebar.
3. Note the matrix: rows = unions, columns = document types, cells = colored status badges.
4. Click any cell to deep-link into that union's detail page.
5. Use the "Status" filter dropdown to show only Overdue unions.

WHAT TO LOOK FOR

→ The overall score per union = (Compliant types) / (required types) × 100. Not-Applicable types are excluded from BOTH numerator and denominator.

→ The "Expiring" status uses the EXPIRING_WINDOW_DAYS constant (90 days) — change the constant in src/lib/auth/constants.ts and every status recomputes immediately.

→ Officer signing in sees the same screen but scoped to only their assigned union — server-side filter, not client-side.

Open compliance matrix →

Edit a compliance rule + preview impact

3 min

Each of the 13 document types has a ComplianceRule: required (bool), renewalYears (how often to re-file), retentionYrs (how long to keep before disposal). Editing a rule recomputes every union's score live — but before saving, you can preview exactly which unions would change status. This is the "preview-impact" pattern: never surprise the user with side effects.

TRY IT

1. Sign in as registrar@example.com.
2. Open Compliance → click "Configure Rules" (top-right).

3. Find "Meeting Minutes" → change renewalYears from 1 to 3.
4. Click "Preview Impact" — a dialog shows which unions would shift from Compliant to Overdue.
5. Cancel or save — the preview is read-only, no side effects until you click Save.

WHAT TO LOOK FOR

→ The preview dialog lists each affected union by name with its current vs. new status — no vague "X unions will be affected", always concrete.

→ After saving, the Compliance matrix updates immediately (live recompute, no cache to invalidate).

→ Audited as "compliance_rule_edit" with before/after diff in the audit log.

Open compliance rules →

Compare unions side-by-side

2 min

The Compare screen lets you pick 2-6 unions and see them side-by-side in a gap matrix: rows = document types, columns = selected unions, cells = colored letter badges (C/E/O/M). Click any cell to jump to that union's detail page. Crucially, Compare does NOT recompute any score — it calls /api/compliance (the same endpoint Compliance uses) and pivots the result client-side.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Compare" in the sidebar.
3. Click "Add Union" → pick 3 unions (e.g. Maritime, Coffee, Stevedores).
4. Read the gap matrix — find types where one union is Compliant and another is Overdue.
5. Try adding a 7th union — the picker blocks at 6 (UI-level block, not submit-level).

WHAT TO LOOK FOR

→ No scoring logic in this screen — single /api/compliance call, client-side pivot. This guarantees Compare never diverges from Compliance.

→ Each cell is a clickable button → /unions/[id]. Try it.

→ The 7th-union block is at the UI level (Add Union button hidden, picker disabled for unselected unions when 6 are picked) — per spec, not a submit-time rejection.

Open compare →

D. Oversight

Phase 5 — Dashboard, Audit, Retention. The Registrar's and Inspector's daily oversight tools.

Start your day on the Dashboard

3 min

The Dashboard is the Registrar's morning command center. Six stat cards summarize the system: total unions, total documents, and compliance summary (compliant/expiring/overdue/missing). Each compliance card deep-links into Compliance pre-filtered by that status — click "Overdue" to jump straight to the overdue list. The Recent Activity feed shows the latest 20 audit entries.

TRY IT

1. Sign in as registrar@example.com (you land here automatically).
2. Read the 6 stat cards across the top.
3. Click the red "Overdue" card → you land on /compliance?status=overdue.
4. Click back → scroll to "Recent Activity" — note your own login is the most recent entry.
5. Scroll to "Documents by Type" — see the distribution across all 13 types.

WHAT TO LOOK FOR

→ The stat cards and activity feed load INDEPENDENTLY — separate useEffects with separate loading states. A slow /api/dashboard/stats doesn't block /api/dashboard/activity.

→ The compliance card deep-links use ?status= as a URL parameter — the Compliance page reads it and pre-selects the filter. This is the loop-close from Phase 3 → Phase 5.

→ Zero-data state: if you reset test data, the cards show "0" and activity shows "No recent activity" — graceful, not broken.

Open dashboard →

Investigate an action in the Audit Log

4 min

Every mutating action across the system writes an AuditLog row with before/after JSON snapshots. The Audit screen lets you filter by entity type, action, actor, and date range — all combine with AND logic. Expand any row to see the diff: Create actions show only "after", Delete actions show only "before", Edits show both side-by-side. Actor names are captured at action time, so even if a user is later renamed or deleted, their audit entries still show the correct name.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Audit Log" in the sidebar.
3. Filter by Entity Type = "Union", Action = "Union Created" — see the 6 demo union creates.
4. Click any row to expand → see the "after" panel only (no empty "before" panel for Create actions).
5. Filter by Actor = "System" — see system-triggered entries (lockouts, seed events).
6. Filter by Start Date = a future date → expect 0 results (verifies date filter works).

WHAT TO LOOK FOR

→ The before/after diff viewer: Create shows only "after" (no confusing empty "before" panel), Delete shows only "before". Edits show both side-by-side with red/green color coding.

→ The Actor dropdown includes "System" as a filter option — but the filter matches BOTH actorId=null AND actorName='System', so login-failed entries for nonexistent emails (which also have actorId=null but actorName = the typed email) don't show up under "System".

→ Pagination caps at 100/page — default 50/page. Total count shown.

Open audit log →

Dispose of an expired document

4 min

Retention is the most irreversible action in the system — once a document is marked disposed, it cannot be brought back (the row stays for audit, but it's gone from every active view). The flow: pending-disposal list shows documents past their retention period → Approve requires a two-step confirmation (type the document title exactly) → Reject snoozes for 30 days. Race-safe: if two Registrars click Approve simultaneously, only one succeeds.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Retention" in the sidebar.
3. Note the pending-disposal list — these are docs whose filedAt + retentionYrs ≤ today.
4. Scroll to the "Retention Schedule" table at the bottom — see retentionYrs per type (constitution = 99 = never dispose).
5. Click "Approve" on any pending doc → the dialog requires you to TYPE THE EXACT TITLE to enable Confirm.
6. Type the title → click "Confirm Disposal" → the doc disappears from the list.
7. Open Audit Log → filter by Entity Type = "Retention" → see the disposal entry with severity = Critical.

WHAT TO LOOK FOR

→ The two-step confirm: the Confirm Disposal button is disabled until titleInput.trim() === entry.title.trim(). This is the same pattern GitHub uses for repo deletion.

→ The audit entry is severity = Critical (disposal is irreversible) — retention_reject is severity = Info (snooze is reversible).

→ The race-safe updateMany pattern: if you open two browser tabs and click Approve on the same doc in both, only one succeeds — the other gets 409 "concurrent_modification".

→ Inspector can see the list + Reject, but NOT Approve — the button is hidden via {isRegistrar && ...}.

Open retention →

G. Productivity (Phase 6)

Tasks (FSD-14), Calendar (FSD-15), BJS-01 Daily Sweep, and the notification bell loop-close.

Create a task + cycle its status

3 min

Tasks are to-do items that can be personal, union-scoped, or assigned to a specific user. The status badge is clickable — click it to cycle Pending → In Progress → Completed without opening the edit dialog. Overdue tasks (dueDate in the past + not completed) are flagged with a red OVERDUE badge, live-computed on every render from the API.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Tasks" in the sidebar.
3. Note the 4 summary cards: Total, Active, Overdue, Urgent.
4. Click "New Task" → fill in title + due date + priority → save.
5. Find your task in the list → click its status badge once (Pending → In Progress).
6. Click it again (In Progress → Completed). Note: no edit dialog opened.
7. Create a task with a due date in the past → note the red OVERDUE badge.

WHAT TO LOOK FOR

- → The status badge is a <button> with cursor-pointer — click it and the PATCH fires with only { status: next }, no edit dialog.
- → Optimistic update: the badge flips immediately. If the PATCH fails, it reverts and a toast appears.
- → The OVERDUE badge is rendered when task.overdue is true — a live-computed boolean from the API (dueDate < today AND status != completed). No client-side date math.
- → The summary cards refetch after every status change — the Active/Overdue counts update immediately.

Open tasks →

Officer's scoped task view

3 min

Officers see a NARROWER task list than Registrars: only tasks for their assigned union OR tasks assigned to them personally (even if for a different union). This OR-condition is the subtle part — a task assigned to an Officer by a Registrar, for a DIFFERENT union than the Officer's own, must still be visible. The assignee dropdown is also narrow: an Officer can only assign to themselves or leave it unassigned — the full user list is never fetched.

TRY IT

1. Sign in as registrar@example.com.
2. Create a task assigned to officer@example.com for a DIFFERENT union than Papua Maritime (e.g. Highlands Coffee).
3. Sign out → sign in as officer@example.com.
4. Open Tasks → confirm you see BOTH your own union's tasks AND the cross-union task assigned to you.
5. Click "New Task" → open the assignee dropdown → confirm it contains ONLY "Myself" and "Unassigned" (not a full user list).

Calendar: composed view of tasks + compliance deadlines

4 min

The Calendar composes data from two sources — Tasks (due dates) and Compliance (renewal deadlines) — into a single monthly grid. Four tabs filter the view: All (Tasks + Documents), Documents (all renewal deadlines), Tasks (due dates), and Compliance (only Expiring/Overdue — a strict subset of Documents). Switching tabs triggers a fresh server request, not client-side filtering. Click a Task event to open the edit dialog in place; click a Document event to navigate to that union.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Calendar" in the sidebar.
3. Note the month grid — days with events show colored chips (blue for tasks, amber for Expiring, red for Overdue, slate for Compliant).
4. Switch between the 4 tabs (All / Documents / Tasks / Compliance) — each triggers a distinct network request.
5. Find a day with 3+ events → note the "+N more" badge. Click the day to expand.
6. Click a Task-type event → the CreateEditTaskDialog opens (reused from Tasks, not rebuilt).
7. Click a Document-type event → navigates to that union's detail page.

WHAT TO LOOK FOR

- → The Compliance tab is a STRICT SUBSET of the Documents tab — if they look identical, that's a bug. Documents shows ALL renewal deadlines; Compliance shows only Expiring/Overdue.
- → The All tab = Tasks + Documents only (NOT + Compliance, which would double-count). The counts badge on each tab reflects this.
- → Clicking a Task event reuses the same CreateEditTaskDialog component from the Tasks screen — single source of truth, no duplicate dialog.
- → Day cells with 3+ events show a "+N more" count badge (expandable) rather than cramming entries inline.

Open calendar →

Notification bell (BJS-01 loop-close)

3 min

The notification bell has existed since Phase 1 but nothing populated it until Phase 6's BJS-01 daily sweep. The cron job runs daily (or on-demand via HTTP), checks every union for Expiring/Overdue compliance issues + every overdue task, and creates Notification rows with a status-inclusive dedup key. The bell now displays real notifications, navigates on click, and marks-as-read without a page reload.

TRY IT

1. Sign in as registrar@example.com.
2. Look at the bell icon in the top-right header — you should see a red badge with "14" (14 unread notifications from the daily sweep).
3. Click the bell → dropdown lists the notifications (compliance overdue alerts, each with title + body + relative time).
4. Click a notification → it marks as read (count decrements immediately, no page reload) + navigates to the union's compliance view.
5. Open the bell again → the clicked notification is gone from the list.
6. To run the sweep manually: curl -X POST -H "Authorization: Bearer urm-cron-secret-7a3b9c2d5e8f1a4b6d0c2e4f6a8b0c2d" http://localhost:3000/api/cron/compliance-sweep

WHAT TO LOOK FOR

- → The bell badge shows the unread count (14 for the demo data). Clicking a notification decrements it optimistically — no page reload.
- → The dedup key includes the STATUS value (compliance:{unionId}:{docType}:{status}). When a document escalates from Expiring to Overdue, a NEW notification is created — the original Expiring one stays undismissed. This is the single most important design detail in Phase 6.
- → Running the sweep twice creates 0 new notifications (idempotent — dedup keys prevent duplicates).
- → Inspectors receive ZERO notifications — only Officers + Registrars are notified about compliance issues.

Open dashboard (bell is top-right) →

H. Public Portal (Phase 7)

FSD-17 Public Disclosure Portal + FSD-18 Legislation Library. Fully public — no login required.

Search the public union registry

3 min

The public portal is a distinct, simpler layout — no sidebar, no user menu, no notification bell. A citizen can search the union registry by name or registration number, see only whitelisted fields (unionName, registrationNo, status — no contact info, no fileNumber, no compliance data), and click through to a union's public documents. Rate-limited to 60 requests/minute per IP.

TRY IT

1. Open a fresh browser tab (or sign out).
2. Navigate to /public/unions — no login required.
3. Type "maritime" in the search bar (debounced 300ms).
4. Note the results show only unionName, registrationNo, and status badge — no contact info, no file numbers.
5. Click "Papua Maritime Workers Union" → opens the public detail page.
6. Scroll to the "Public Documents" section — only Certificate + Constitution appear (other types are internal-only).

WHAT TO LOOK FOR

→ The public layout has NO authenticated chrome — no sidebar, no user menu, no notification bell. Just a minimal header with "Union Records Portal" + nav links.

→ The search is debounced 300ms — no request fires on every keystroke.

→ A deregistered union (Western Province Miners League) still appears in public search with its history intact — deregistering doesn't hide public disclosure.

→ Inspect the raw API response (browser devtools → Network): the union object has EXACTLY 4 keys (id, unionName, registrationNo, status). No fileNumber, no address, no contactPerson.

Open public search →

Download a public document (security-checked)

2 min

The public file download endpoint (/api/public/files/[id]) is the highest-risk endpoint in the system. A fully anonymous caller must never retrieve a non-public document type or a disposed one. Every

Legislation Library (client-side filtering)

2 min

The Legislation Library is a static config file (no database) bundled with the app. At startup, the app validates every referenced file exists on disk — if any are missing, the app FAILS to start (loud error naming the specific file). The UI filters client-side (no network request on filter) because the dataset is small + static.

TRY IT

1. Navigate to /public/legislation — no login required.
2. Note the 5 legislation documents (Industrial Relations Act, Trade Unions Regulation, etc.).
3. Type "industrial" in the search bar → the list filters instantly (no network request — check devtools).
4. Click a category chip (e.g. "Primary Legislation") → filters by category, also client-side.
5. Click "Download" on any document → the PDF downloads.

WHAT TO LOOK FOR

- → Filtering happens with NO network request — open browser devtools → Network tab → type in the search bar → no new requests fire. The filtering is 100% client-side via useMemo.
- → Two distinct empty states: "No legislation documents are currently available" (zero entries) vs "No documents match your search" (no matches).
- → The startup validation: if a referenced PDF is missing from disk, the app refuses to start. The error message names the specific missing file. (Verified in Chunk 7.4 — temporarily remove a file + restart to see it.)
- → No database model for legislation — it's a static JSON config file at public/legislation/legislation.json. Changes are a deployment, not a runtime update.

Open legislation library →

E. Administration

Phase 4 — Users, Security, API Keys. Registrar-only screens for system administration.

Manage users + last-Registrar protection

4 min

The Users screen is Registrar-only. Create users (no password at creation — a set-password email is sent), edit roles, disable, enable, reset 2FA, reset password. Three protection mechanisms work independently: lockout (3 failed attempts), disabled (admin toggle), and 2FA reset (clears the secret). The last-Registrar protection prevents you from disabling or role-changing the only remaining active Registrar — system-wide count, not self-check.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Users" in the sidebar — see the 3 seeded users.
3. Try to disable yourself → blocked with "You cannot disable the last remaining Registrar".
4. Try to change your own role to Inspector → blocked with the same message.
5. Reset 2FA on the officer account → the secret is cleared (the user must set up 2FA again).
6. Reset password on the officer account → an email is sent; the API NEVER accepts a plaintext password in the request body.

WHAT TO LOOK FOR

→ Three-mechanism independence: Unlock clears ONLY failedLoginAttempts + lockedUntil. Disable sets disabled=true but doesn't touch lockout. 2FA reset clears twoFactorSecret but doesn't touch either. They can all coexist on the same user.

→ Last-Registrar protection is a SYSTEM-WIDE count of OTHER active Registrars — not a self-check. So you can't disable yourself if you're the only one, but you CAN if there are 2+ active Registrars.

→ Password reset NEVER accepts a plaintext password in the request body — the API generates a token, sends an email,

Review security overview + unlock a locked account

3 min

The Security screen is the Registrar's early-warning dashboard. Cards show: failed logins in last 24h / 7d, currently locked-out accounts, disabled accounts, 2FA adoption rate (excluding disabled users from the denominator), active sessions. Below the cards: a filterable login-attempts table and a locked-out users list with one-click Unlock.

TRY IT

1. Sign in as registrar@example.com.
2. Click "Security" in the sidebar.
3. Read the 6 overview cards — note 2FA adoption rate excludes disabled users.
4. To generate lockout data: open a private window, try logging in as inspector@example.com with a wrong password 3 times.
5. Refresh Security → the "Locked-Out Users" card should show 1.
6. Click "Unlock" on the locked inspector account — note it clears ONLY the lockout (not disabled, not 2FA, not password).
7. The inspector can now attempt login again immediately.

WHAT TO LOOK FOR

→ The "Also Disabled" badge: if a user is BOTH locked out AND disabled, the badge appears — visual confirmation that the two states coexist independently.

→ 2FA adoption rate denominator is `db.user.count({ where: { disabled: false } })` — disabled users excluded from BOTH numerator and denominator.

→ The Unlock response includes `disabledUnchanged: true` if the user was also disabled — so the UI can show "Note: this user is also disabled" rather than implying unlock = full restore.

Open security →

Create an API key + test it

4 min

API keys enable programmatic access (scripts, integrations) without a browser session. Created by Registrars, scoped to a fixed set of permissions (unions/documents/compliance/audit — deliberately NO users:* or api-keys:* scope), max 365-day expiry. The full key value is shown EXACTLY ONCE on creation — only the SHA-256 hash + first-12-char prefix are persisted. Revocation is a soft-delete (revokedAt set) for audit history.

TRY IT

1. Sign in as registrar@example.com.
2. Click "API Keys" in the sidebar.
3. Click "Create Key" → name it "Test integration", pick scopes (unions:read, documents:read), set expiry to 30 days.
4. On the create-success dialog, COPY the full key value (starts with urm_) — this is the only time it's shown.
5. In a terminal: curl -H "Authorization: Bearer YOUR_KEY" http://localhost:3000/api/unions → should return the union list.
6. Back in the UI, click "Revoke" on the key → confirm. Try the curl again → 401 Unauthorized.
7. Note the revoked key is still VISIBLE in the list (with a "Revoked" badge) — soft-delete for audit.

WHAT TO LOOK FOR

→ The full key value is shown ONCE. Subsequent GET /api/api-keys returns only the prefix (e.g. "urm_a1b2c3d4..."), never the full value, never the keyHash.

→ PATCH /api/api-keys/[id] rejects scope/expiry changes with "Revoke and reissue" — scopes are fixed at creation.

→ Loop-close from Chunk 4.2: if you DISABLE the key's owner, the key immediately stops authenticating (401). Re-enable the owner → key works again. This is the disabled-user-API-key check built into auth-guard.ts.

Open API keys →

F. Officer's scoped view

Sign in as the union_officer role to see the system from the union's perspective — scoped, restricted, practical.

Sign in as a Union Officer

3 min

The Officer role represents a union's appointed representative. They see ONLY their own union (My Union, not the global Unions registry), their own compliance (scoped server-side), and have NO access to Documents (global search), Compare, Users, Security, API Keys, or Audit. They CAN edit their own union's contact fields (address, phone, email) but NOT unionName, registrationNo, fileNumber, status, or remarks — those are silently pinned server-side.

TRY IT

1. Sign out, then sign in as officer@example.com / Test!2345.
2. Note the nav sidebar: only "My Union", "Compliance", "Dashboard". (Tasks and Calendar are Phase 6 placeholders.)
3. Click "My Union" — you see Papua Maritime Workers Union (assigned in demo data).
4. Try editing the address field → saves successfully.
5. Try editing unionName via the network tab (curl with a custom body) → the API silently ignores it; the response reflects the original name.
6. Click "Compliance" — you see ONLY your union's compliance, not the system-wide matrix.

WHAT TO LOOK FOR

→ The nav excludes Unions, Documents, Compare, Audit, Users, Security, API Keys, Retention — Officer has no oversight role.

→ The Officer's PUT /api/unions/[id] request silently pins restricted fields (unionName, registrationNo, fileNumber, status, remarks) to their existing values. The response reflects the actual state, proving the pin worked — not a 403 rejection.

→ Compliance scope: /api/compliance returns only the officer's union when the role is union_officer — server-side filter, not client-side. Trying to bypass with ?unionId=other-union returns the officer's own union anyway.

Sign out + sign in as officer →

Build complete

All 7 phases are built — 37/37 chunks complete. The system is a full Union Records Management System spanning authentication, records, compliance, admin, oversight, productivity, and public disclosure. Ready for migration to server.

14 Prisma models | 38 verify scripts (700+ assertions) | 13 nav items | 3 roles + public portal